```
1  -- 16-bit Q1.14 Cordic Calculator;
2  --
3  --  This entity describes a 16 bit Cordic calculator (1 integer bit and 14
4  --  decimal bits.) This calculator implements cos, sin, cosh, sinh, division,
5  --  and multiplication fully parallely. This implementation has three pipeline
6  --  stages, such that each 4 cordic stages are operating on a different function
7  --  at a given clock cycle.
8  --
9  --  Revision History:
10 --    Date:         Author:    Description:
11 --    07 Jan 2024   Chris M.   Initial revision; described entity and processes.
12 --    24 Jan 2025   Chris M.   Used 22-bit adder subtracters instead of
13 --                             directly insantiating them to remove nested
14 --                             generate loops.
15 --    25 Jan 2025   Chris M.   Added mode signal.
16 --    26 Jan 2025   Chris M.   Added comments.
17 --    27 Jan 2025   Chris M.   Fixed bug where the initial value of the Y
18 --                             adders was being set incorrectly (inital values
19 --                             go to the A input and not the B input)
20 --    27 Jan 2025   Chris M.   Changed lookup table to be generated locally.
21 --    31 Jan 2025   Chris M.   Moved fixed point operation functions to
22 --                             seperate file for reuse in test bench.
23 --    31 Jan 2025   Chris M.   Removed reset signal.
24 --    18 Feb 2025   Chris M.   Formatted. Add [TODO].
25 --    18 Feb 2025   Chris M.   Converted to use CordicStage
26 --    21 Feb 2025   Chris M.   Fixed bugs in linear and hyperbolic functions.
27 --    21 Feb 2025   Chris M.   Corrected timing.
28 --    25 Feb 2025   Chris M.   Added 1 pipeline stage.
29 --    25 Feb 2025   Chris M.   Modified for 3 pipeline stages.
30 --    25 Feb 2025   Chris M.   Added documentation.
31 --
32 -- TODO:
33 --    - Remove magic numbers from for generate loops.
34 --    - See if Xilinix supports if/generate
35 ----------------------------------------------------------------------------
36
37 -- libraries
38 library  ieee;
39 library osvvm;
40
41 use  std.textio.all;
42 use  ieee.std_logic_1164.all;
43 use  ieee.numeric_std.all;
44 use  ieee.math_real.all;
45 use  work.FixedLib.all;
46 use  work.CordicConstants.all;
47 use  work.all;
48 use  ANSIEscape.all;
49
50 use osvvm.RandomPkg.all;
51 use osvvm.CoveragePkg.all;
52 context osvvm.OsvvmContext;
53
54 entity Cordic is
```

```
55   port(
56     CLK   : in  std_logic;
57     X     : in  std_logic_vector(15 downto 0);
58     Y     : in  std_logic_vector(15 downto 0);
59     Func  : in  std_logic_vector( 4 downto 0);
60     R     : out std_logic_vector(15 downto 0)
61   );
62 end Cordic;
63
64 architecture sim of Cordic is
65
66   -- Constants and Types ---------------------------------------------------------------------
67
68
69   -- Signals and Constants -------------------------------------------------------------------
70
71   -- The latched inputs
72   signal X_temp    : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
73   signal Y_temp    : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
74   signal Func_temp : std_logic_vector(Func'range);
75
76
77   -- Pipelined outputs/inputs. Note that these go between the 7th (8th if counting from 1) cordic
78   -- stage.
79
80   -- First stage.
81   signal XPipeline1 : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
82   signal YPipeline1 : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
83   signal ZPipeline1 : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
84
85   signal FuncPipeline1       : std_logic_vector(Func'range);
86   signal CordSystemPipeline1 : std_logic_vector(1 downto 0);
87   signal ModePipeline1       : std_logic;
88
89   -- Second stage.
90   signal XPipeline2 : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
91   signal YPipeline2 : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
92   signal ZPipeline2 : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
93
94   signal FuncPipeline2       : std_logic_vector(Func'range);
95   signal CordSystemPipeline2 : std_logic_vector(1 downto 0);
96   signal ModePipeline2       : std_logic;
97
98   -- Third stage.
99   signal XPipeline3 : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
100  signal YPipeline3 : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
101  signal ZPipeline3 : std_logic_vector(INTERMEDIATE_WORD_SIZE - 1 downto 0);
102
103  signal FuncPipeline3       : std_logic_vector(Func'range);
104  signal CordSystemPipeline3 : std_logic_vector(1 downto 0);
105  signal ModePipeline3       : std_logic;
106
107
108  -- Start and end indices for cordic sections. The cordic is divided into 4 sections:
```

```
109  --    0 - 3, 4 - 7, 8 - 11, and 12 - 15
110  -- There is a DFF between each section as well as the first and last section
111  -- for I/O.
112
113  constant FIRST_QUARTER_START  : natural := 0;
114  constant FIRST_QUARTER_END    : natural := 3;
115  constant SECOND_QUARTER_START : natural := 4;
116  constant SECOND_QUARTER_END   : natural := 7;
117  constant THIRD_QUARTER_START  : natural := 8;
118  constant THIRD_QUARTER_END    : natural := 11;
119  constant FOURTH_QUARTER_START : natural := 12;
120  constant FOURTH_QUARTER_END   : natural := 15;
121
122
123  -- Cordic modes. Used to change the function of the cordic. In circular
124  -- coordinates, rotation mode is used for trig functions and vectoring mode
125  -- is used for inverse trig functions. In linear mode, rotation mode is used
126  -- to get x*z and vectoring mode is used to get y/x. In hyperbolic mode,
127  -- rotation mode is used to find hyperbolic trig functions and vectoring mode
128  -- is used to find their inverses.
129  signal   Mode      : std_logic;
130  constant ROTATION  : std_logic := '0';
131  constant VECTORING : std_logic := '1';
132
133  -- Result busses for the 22-bit adders
134  signal XResultBus : array_2d;
135  signal YResultBus : array_2d;
136  signal ZResultBus : array_2d;
137
138  signal ResultComb : std_logic_vector(WORD_SIZE - 1 downto 0);
139
140  -- Wtf ???? This wasn't declared before but it still worked ????
141  signal CordSystem : std_logic_vector(1 downto 0);
142
143  -- Input busses for the 22-bit adders. Note that InputBus(k) is just
144  -- ResultBus(k-1). InputBus(0) depends on the function. Also note that all of
145  -- the b inputs for the Z-adders are fixed.
146  signal XInputBus : array_2d;
147  signal YInputBus : array_2d;
148  signal ZInputBus : array_2d;
149
150  signal CordicLogID       : AlertLogIDType := GetAlertLogID("Cordic");
151
152  -- The decision variables. If in rotation mode, it implies adding when the
153  -- current angle is less than the target or subtraction when the current
154  -- angle is more than the target. If in vectoring mode, d implies addition
155  -- if the sign of y is positive and subtraction if the sign of y is negative.
156  signal DBus : std_logic_vector(WORD_SIZE - 1 downto 0);
157
158  signal ZConstantBus : array_2d;
159
160  -- Functions and Procedures ---------------------------------------------------------------
161
162  -- Repeat a character n times and return a string.
```

```
163  function RepeatChar(c : character; n : positive) return string is
164    variable Result : string(1 to n) := (others => ' ');
165  begin
166    for i in 1 to n loop
167      Result(i) := c;
168    end loop;
169    return Result;
170  end function;
171
172  -- Get the MSB of a std_logic_vector.
173  function GetMSB(slv : std_logic_vector) return std_logic is
174  begin
175    return slv(slv'high);
176  end function;
177
178  -- Convert a Q3.18 fixed point to a Q1.14 fixed point.
179  function Q3_18_To_Q1_14(slv : std_logic_vector(21 downto 0)) return std_logic_vector is
180  begin
181    return slv(21) & slv(18) & slv(17 downto 4);
182  end function;
183
184  -- Print the current function to the debug stream
185  procedure PrintFunc(Xin : std_logic_vector(WORD_SIZE - 1 downto 0);
186                      Yin : std_logic_vector(WORD_SIZE - 1 downto 0);
187                      FuncIn : std_logic_vector(Func'range)) is
188  begin
189    case FuncIn is
190      when F_COS_X =>
191        Log(CordicLogID, "cos(" & to_string(Q1_14ToReal(XIn)) & ")", DEBUG);
192      when F_SIN_X =>
193        Log(CordicLogID, "sin(" & to_string(Q1_14ToReal(XIn)) & ")", DEBUG);
194      when F_COSH_X =>
195        Log(CordicLogID, "cosh(" & to_string(Q1_14ToReal(XIn)) & ")", DEBUG);
196      when F_SINH_X =>
197        Log(CordicLogID, "sinh(" & to_string(Q1_14ToReal(XIn)) & ")", DEBUG);
198      when F_X_MULT_Y =>
199        LOG(CordicLogID, to_string(Q1_14ToReal(XIn)) & " * " & to_string(Q1_14ToReal(YIn)), DEBUG);
200      when F_Y_DIV_X =>
201        Log(CordicLogID, to_string(Q1_14ToReal(YIn)) & " / " & to_string(Q1_14ToReal(XIn)), DEBUG);
202      when others =>
203        Log(CordicLogID, "Unknown function: " & to_string(FuncIn), DEBUG);
204    end case;
205  end procedure;
206
207  procedure PrintFuncTemp(FuncIn : std_logic_vector(Func'range)) is
208  begin
209    case FuncIn is
210      when F_COS_X =>
211        Log(CordicLogID, "Func_temp: cos", DEBUG);
212      when F_SIN_X =>
213        Log(CordicLogID, "Func_temp: sin", DEBUG);
214      when F_COSH_X =>
215        Log(CordicLogID, "Func_temp: cosh", DEBUG);
216      when F_SINH_X =>
```

```
217         Log(CordicLogID, "Func_temp: sinh", DEBUG);
218       when F_X_MULT_Y =>
219         LOG(CordicLogID,  "Func_temp: x * y", DEBUG);
220       when F_Y_DIV_X =>
221         Log(CordicLogID,  "Func_temp x / y", DEBUG);
222       when others =>
223         Log(CordicLogID, "Func_temp: Unknown function: " & to_string(FuncIn), DEBUG);
224     end case;
225   end procedure;
226 
227 begin
228 
229   SetLogEnable(CordicLogID, DEBUG, true);
230 
231   -- Load the input and output values based on the operation. The starting
232   -- and final X Y Z vectors are based on the following table:
233   -- +==================+======================+================================+
234   -- | Coordinate System| Rotation Mode        | Vectoring Mode                 |
235   -- +==================+======================+================================+
236   -- | Circular         | ⌜     ⌝    ⌜     ⌝   | ⌜     ⌝    ⌜      ________ ⌝ |
237   -- | m = 1            | | K ⁻¹|    |cos θ|   | |  x  |    | K⁻¹ √ x² + y² | |
238   -- |                  | | 0   | => |sin θ|   | |  y  | => |       0       | |
239   -- |                  | |theta|    |  0  |   | |theta|    | tan⁻¹(y/x)    | |
240   -- |                  | ⌞     ⌟    ⌞     ⌟   | ⌞     ⌟    ⌞               ⌟ |
241   -- +------------------+----------------------+--------------------------------+
242   -- | linear           | ⌜   ⌝    ⌜     ⌝     | ⌜   ⌝    ⌜     ⌝             |
243   -- | m = 0            | | x |    |  x  |     | | x |    |  x  |             |
244   -- |                  | | 0 | => | x*z |     | | y | => |  0  |             |
245   -- |                  | | z |    |  0  |     | | 0 |    | y/x |             |
246   -- |                  | ⌞   ⌟    ⌞     ⌟     | ⌞   ⌟    ⌞     ⌟             |
247   -- +------------------+----------------------+--------------------------------|
248   -- | Hyperbolic       | ⌜     ⌝    ⌜      ⌝   | ⌜     ⌝    ⌜      ________ ⌝ |
249   -- | m = -1           | | K⁻¹ |    |cosh θ|   | |  x  |    | K⁻¹ √ x² + y² | |
250   -- |                  | | 0   | => |sinh θ|   | |  y  | => |       0       | |
251   -- |                  | |theta|    |  0   |   | |theta|    | tanh⁻¹(y/x)   | |
252   -- |                  | ⌞     ⌟    ⌞      ⌟   | ⌞     ⌟    ⌞               ⌟ |
253   -- +------------------+----------------------+--------------------------------+
254   --
255   -- Note that theta is the 'x' input, since we have x and y inputs (these are
256   -- not the same as the components of the (x y z) vector).
257 
258   -- X adder inputs
259   with Func_temp select XInputBus(0) <=
260     (K_inv_circular)    when F_COS_X    | F_SIN_X,
261     (K_inv_hyperbolic)  when F_COSH_X   | F_SINH_X,
262     (X_temp)            when F_X_MULT_Y | F_Y_DIV_X,
263     (XInputBus(0))      when others;
264 
265   -- Y adder inputs
266   with Func_temp select YInputBus(0) <=
267     (others => '0')   when F_COS_X  | F_SIN_X | F_X_MULT_Y | F_COSH_X | F_SINH_X,
268     (Y_temp)          when F_Y_DIV_X,
269     (others => '0')   when others;
270 
```

```
271  -- Z adder inputs
272  with Func_temp select ZInputBus(0) <=
273    (X_temp)          when F_COS_X | F_SIN_X | F_COSH_X | F_SINH_X,
274    (Y_temp)          when F_X_MULT_Y,
275    (others => '0')   when F_Y_DIV_X,
276    (others => '0')   when others;
277
278
279  ------------------------------------------------------------------------------------------------
280  -- ZConstantBus generation -----------------------------------------------------------------
281  ------------------------------------------------------------------------------------------------
282
283  -- Generate the ZConstants for the first quarter of the Cordic. They depend on
284  -- Func_temp.
285  ZConstantBusGen_FirstQuarter:
286  --       0 to 3
287  for i in FIRST_QUARTER_START to FIRST_QUARTER_END generate
288    with Func_temp select ZConstantBus(i) <=
289      Z_CONSTANTS_CIRCULAR(i)   when F_COS_X    | F_SIN_X,
290      Z_CONSTANTS_LINEAR(i)     when F_X_MULT_Y | F_Y_DIV_X,
291      Z_CONSTANTS_HYPERBOLIC(i) when F_COSH_X   | F_SINH_X,
292      (others => '0')           when others;
293  end generate;
294
295  -- Generate the ZConstants for the second quarter of the Cordic. They depend on
296  -- FuncPipeline1.
297  ZConstantBusGen_SecondQuarter:
298  --       4 to 7
299  for i in SECOND_QUARTER_START to SECOND_QUARTER_END generate
300    with FuncPipeline1 select ZConstantBus(i) <=
301      Z_CONSTANTS_CIRCULAR(i)   when F_COS_X    | F_SIN_X,
302      Z_CONSTANTS_LINEAR(i)     when F_X_MULT_Y | F_Y_DIV_X,
303      Z_CONSTANTS_HYPERBOLIC(i) when F_COSH_X   | F_SINH_X,
304      (others => '0')           when others;
305  end generate;
306
307  -- Generate the ZConstants for the third quarter of the Cordic. They depend on
308  -- FuncPipeline2.
309  ZConstantBusGen_ThirdQuarter:
310  --       8 to 11
311  for i in THIRD_QUARTER_START to THIRD_QUARTER_END generate
312    with FuncPipeline2 select ZConstantBus(i) <=
313      Z_CONSTANTS_CIRCULAR(i)   when F_COS_X    | F_SIN_X,
314      Z_CONSTANTS_LINEAR(i)     when F_X_MULT_Y | F_Y_DIV_X,
315      Z_CONSTANTS_HYPERBOLIC(i) when F_COSH_X   | F_SINH_X,
316      (others => '0')           when others;
317  end generate;
318
319  -- Generate the ZConstants for the fourth quarter of the Cordic. They depend on
320  -- FuncPipeline3.
321  ZConstantBusGen_FourthQuarter:
322  --       12 to 15
323  for i in FOURTH_QUARTER_START to FOURTH_QUARTER_END generate
324    with FuncPipeline3 select ZConstantBus(i) <=
```

```
      Z_CONSTANTS_CIRCULAR(i)   when F_COS_X    | F_SIN_X,
      Z_CONSTANTS_LINEAR(i)     when F_X_MULT_Y | F_Y_DIV_X,
      Z_CONSTANTS_HYPERBOLIC(i) when F_COSH_X   | F_SINH_X,
      (others => '0')           when others;
  end generate;


  ------------------------------------------------------------------------------------------------
  -- DBus Generation -----------------------------------------------------------------------------
  ------------------------------------------------------------------------------------------------

  -- When we are in vectoring mode, we want to drive y towards zero. Note that the
  -- YAddBarSub signal is just d_i, so we want to pick d_i such that the operation
  -- goes towards zero:
  --
  -- If the YInput is negative, then pick the operation such that the result will
  -- become more positive. The B input for the Y adder is just the XInput
  -- divided by 2, so the sign must be the same.
  --
  -- +===+===+====+
  -- | Y | X | Op |
  -- +===+===+====+
  -- | - | - |  - |
  -- | - | + |  + |
  -- | + | - |  + |
  -- | + | + |  - |
  -- +------------+
  -- Note that this just results in XORing DBus signal by the sign of XInput

  -- Generate the DBus signals for the first quarter of the Cordic. They depend on
  -- Mode.
  DBusGen_FirstQuarter :
  --       0 to 3
  for i in FIRST_QUARTER_START to FIRST_QUARTER_END generate
    DBus(i) <=
      -- Add if in vectoring mode and Y is negative and X is positive.
      -- Subtract if in vectoring mode and Y is negative and X is negative.
      '0' xor GetMSB(XInputBus(i)) when ((Mode = VECTORING) and
                                         (signed(YInputBus(i)) < signed(ZERO_22))) else

      -- Subtract if in vectoring mode and Y is positive and X is positive.
      -- Add if in vectoring mode and Y is negative and X is positive
      '1' xor GetMSB(XInputBus(i)) when ((Mode = VECTORING) and (YInputBus(0)(21) = '0')) else

      -- Add if in rotation mode and the input Z is negative
      '1' when ((Mode = ROTATION)  and (signed(ZInputBus(i)) < signed(ZERO_22))) else

      -- Subtract if in rotation mode and the input Z is positive
      '0' when ((Mode = ROTATION) and ZInputBus(0)(21) = '0') else

      -- Set to zero
      '0';
  end generate;

```

```
379  -- Generate the DBus signals for the second quarter of the Cordic. They depend on ModePipeline1.
380  DBusGen_SecondQuarter :
381  --       4 to 7
382  for i in SECOND_QUARTER_START to SECOND_QUARTER_END generate
383    DBus(i) <=
384      '0' xor GetMSB(XInputBus(i)) when ((ModePipeline1 = VECTORING) and
385                                         (signed(YInputBus(i)) < signed(ZERO_22))) else
386
387      '1' xor GetMSB(XInputBus(i)) when ((ModePipeline1 = VECTORING) and (YInputBus(0)(21) = '0')) else
388
389      '1' when ((ModePipeline1 = ROTATION)  and (signed(ZInputBus(i)) < signed(ZERO_22))) else
390
391      '0' when ((ModePipeline1 = ROTATION) and ZInputBus(0)(21) = '0') else
392
393      '0';
394   end generate;
395
396  -- Generate the DBus signals for the third quarter of the Cordic. They depend on ModePipeline2.
397  DBusGen_ThirdQuarter :
398  --       8 to 11
399  for i in THIRD_QUARTER_START to THIRD_QUARTER_END generate
400    DBus(i) <=
401      '0' xor GetMSB(XInputBus(i)) when ((ModePipeline2 = VECTORING) and
402                                         (signed(YInputBus(i)) < signed(ZERO_22))) else
403
404      '1' xor GetMSB(XInputBus(i)) when ((ModePipeline2 = VECTORING) and (YInputBus(0)(21) = '0')) else
405
406      '1' when ((ModePipeline2 = ROTATION)  and (signed(ZInputBus(i)) < signed(ZERO_22))) else
407
408      '0' when ((ModePipeline2 = ROTATION) and ZInputBus(0)(21) = '0') else
409
410      '0';
411   end generate;
412
413  -- Generate the DBus signals for the fourth quarter of the Cordic. They depend on ModePipeline3.
414  DBusGen_FourthQuarter :
415  --       12 to 15
416  for i in FOURTH_QUARTER_START to FOURTH_QUARTER_END generate
417    DBus(i) <=
418      '0' xor GetMSB(XInputBus(i)) when ((ModePipeline3 = VECTORING) and
419                                         (signed(YInputBus(i)) < signed(ZERO_22))) else
420
421      '1' xor GetMSB(XInputBus(i)) when ((ModePipeline3 = VECTORING) and (YInputBus(0)(21) = '0')) else
422
423      '1' when ((ModePipeline3 = ROTATION)  and (signed(ZInputBus(i)) < signed(ZERO_22))) else
424
425      '0' when ((ModePipeline3 = ROTATION) and ZInputBus(0)(21) = '0') else
426
427      '0';
428   end generate;
429
430  ------------------------------------------------------------------------------------------------
431  -- Input Bus Generation --------------------------------------------------------------------------
432  ------------------------------------------------------------------------------------------------
```

```
433
434   -- Generate the input bus for the first quarter of the Cordic (0 - 3). Note that
435   -- InputBus(0) is set by the function.
436   InputBusGen_FirstQuarter :
437   --       1 to 3
438   for i in FIRST_QUARTER_START + 1 to FIRST_QUARTER_END generate
439     XInputBus(i) <= XResultBus(i-1);
440     YInputBus(i) <= YResultBus(i-1);
441     ZInputBus(i) <= ZResultBus(i-1);
442   end generate;
443
444
445   -- Generate the input bus for the second quarter of the Cordic (4 to 7).
446   -- The input to stage 4 for is XPipeline1.
447   --        4
448   XInputBus(SECOND_QUARTER_START) <= XPipeline1;
449   YInputBus(SECOND_QUARTER_START) <= YPipeline1;
450   ZInputBus(SECOND_QUARTER_START) <= ZPipeline1;
451
452   InputBusGen_SecondQuarter :
453   --       5 to 7
454   for i in SECOND_QUARTER_START + 1 to SECOND_QUARTER_END generate
455     XInputBus(i) <= XResultBus(i-1);
456     YInputBus(i) <= YResultBus(i-1);
457     ZInputBus(i) <= ZResultBus(i-1);
458   end generate;
459
460
461   -- Generate the input bus for the third quarter of the Cordic (8 to 11).
462   -- The input to stage 8 for is XPipeline2.
463   --        8
464   XInputBus(THIRD_QUARTER_START) <= XPipeline2;
465   YInputBus(THIRD_QUARTER_START) <= YPipeline2;
466   ZInputBus(THIRD_QUARTER_START) <= ZPipeline2;
467
468   InputBusGen_ThirdQuarter :
469   --       9 to 11
470   for i in THIRD_QUARTER_START + 1 to THIRD_QUARTER_END generate
471     XInputBus(i) <= XResultBus(i-1);
472     YInputBus(i) <= YResultBus(i-1);
473     ZInputBus(i) <= ZResultBus(i-1);
474   end generate;
475
476
477   -- Generate the input bus for the third quarter of the Cordic (12 to 15).
478   -- The input to stage 12 for is XPipeline3.
479   --        12
480   XInputBus(FOURTH_QUARTER_START) <= XPipeline3;
481   YInputBus(FOURTH_QUARTER_START) <= YPipeline3;
482   ZInputBus(FOURTH_QUARTER_START) <= ZPipeline3;
483
484   InputBusGen_FourthQuarter :
485   --       13 to 15
486   for i in FOURTH_QUARTER_START + 1 to FOURTH_QUARTER_END generate
```

```
487     XInputBus(i) <= XResultBus(i-1);
488     YInputBus(i) <= YResultBus(i-1);
489     ZInputBus(i) <= ZResultBus(i-1);
490   end generate;
491
492
493   ---------------------------------------------------------------------------------------------
494   -- Cordic Stages Generation -----------------------------------------------------------------
495   ---------------------------------------------------------------------------------------------
496
497   -- Generate the CordicStage instances for the first quarter.
498
499   -- Note that stage 3 outputs to ResultBus(3), and the pipeline DFFs are set to ResultBus(3) on
500   -- the rising edge of the clock. These CordicStages depend on CordSystem.
501   CordicStageGen_FirstQuarter :
502   --       0 to  3.
503   for i in FIRST_QUARTER_START to FIRST_QUARTER_END generate
504     CordicStage_i : entity CordicStage
505       generic map (
506         wordsize => INTERMEDIATE_WORD_SIZE,
507         i        => i
508       )
509       port map (
510         XIn        => XInputBus(i),
511         YIn        => YInputBus(i),
512         ZIn        => ZInputBus(i),
513         ZConstant  => ZConstantBus(i),
514         CordSystem => CordSystem,
515         d          => DBus(i),
516         XOut       => XResultBus(i),
517         YOut       => YResultBus(i),
518         ZOut       => ZResultBus(i)
519       );
520   end generate;
521
522
523
524   -- Generate the CordicStage instances for the second quarter.
525   -- Note that InputBus(4) is set to the pipeline1 signals on the rising edge of the clock, and
526   -- CordicStage7 outputs to pipeline2 signals. These cordic stages depend on CordSystemPipeline1.
527   CordicStageGen_SecondQuarter :
528   --       4 to 7.
529   for i in SECOND_QUARTER_START to SECOND_QUARTER_END generate
530     CordicStage_i : entity CordicStage
531       generic map (
532         wordsize => INTERMEDIATE_WORD_SIZE,
533         i        => i
534       )
535       port map (
536         XIn        => XInputBus(i),
537         YIn        => YInputBus(i),
538         ZIn        => ZInputBus(i),
539         ZConstant  => ZConstantBus(i),
540         CordSystem => CordSystemPipeline1,
```

```
541         d          => DBus(i),
542         XOut       => XResultBus(i),
543         YOut       => YResultBus(i),
544         ZOut       => ZResultBus(i)
545       );
546   end generate;
547
548
549
550   -- Generate the CordicStage instances for the third quarter.
551   -- Note that InputBus(8) is set to the pipeline2 signals on the rising edge of the clock, and
552   -- CordicStage11 outputs to pipeline3 signals. These cordic stages depend on CordSystemPipeline2.
553   CordicStageGen_ThirdQuarter :
554   --       8 to 11.
555   for i in THIRD_QUARTER_START to THIRD_QUARTER_END generate
556     CordicStage_i : entity CordicStage
557       generic map (
558         wordsize => INTERMEDIATE_WORD_SIZE,
559         i        => i
560       )
561       port map (
562         XIn        => XInputBus(i),
563         YIn        => YInputBus(i),
564         ZIn        => ZInputBus(i),
565         ZConstant  => ZConstantBus(i),
566         CordSystem => CordSystemPipeline2,
567         d          => DBus(i),
568         XOut       => XResultBus(i),
569         YOut       => YResultBus(i),
570         ZOut       => ZResultBus(i)
571       );
572   end generate;
573
574
575
576   -- Generate the CordicStage instances for the fourth quarter.
577   -- Note that InputBus(12) is set to the pipeline3 signals on the rising edge of the clock, and
578   -- CordicStage15 outputs to the actual outputs.
579   CordicStageGen_FourthQuarter :
580   --       12 to 15.
581   for i in FOURTH_QUARTER_START to FOURTH_QUARTER_END generate
582     CordicStage_i : entity CordicStage
583       generic map (
584         wordsize => INTERMEDIATE_WORD_SIZE,
585         i        => i
586       )
587       port map (
588         XIn        => XInputBus(i),
589         YIn        => YInputBus(i),
590         ZIn        => ZInputBus(i),
591         ZConstant  => ZConstantBus(i),
592         CordSystem => CordSystemPipeline3,
593         d          => DBus(i),
594         XOut       => XResultBus(i),
```

```
595         YOut       => YResultBus(i),
596         ZOut       => ZResultBus(i)
597       );
598   end generate;
599
600
601   -- Note that these internal signals are set in the first half of the cordic and stored for the
602   -- second half of the calculation.
603   with Func_temp select Mode <=
604     ROTATION        when F_COS_X | F_SIN_X | F_X_MULT_Y | F_COSH_X | F_SINH_X,
605     VECTORING       when F_Y_DIV_X,
606     '0'             when others;
607
608   with Func_temp select CordSystem <=
609     CIRCULAR        when   F_COS_X    | F_SIN_X,
610     LINEAR          when   F_X_MULT_Y | F_Y_DIV_X,
611     HYPERBOLIC      when   F_COSH_X   | F_SINH_X,
612     (others => '0') when   others;
613
614   -- The result is determined by the last quarter of the cordic (FuncPipeline3)
615   with FuncPipeline3 select ResultComb <=
616     -- Preserve the sign bit
617     Q3_18_To_Q1_14(XResultBus(15)) when F_COS_X | F_COSH_X,
618     Q3_18_To_Q1_14(YResultBus(15)) when F_SIN_X | F_X_MULT_Y | F_SINH_X,
619     Q3_18_To_Q1_14(ZResultBus(15)) when F_Y_DIV_X,
620     (others => '0')                when others;
621
622
623   -- Set the result on the rising edge.
624   process(clk)
625   begin
626     if (rising_edge(clk)) then
627       -- SetLogEnable(CordicLogID, DEBUG, true);
628       -- SetLogEnable(CordicLogID, DEBUG, false);
629       R <= ResultComb;
630     end if;
631   end process;
632
633   -- Latch the inputs and set the pipelined values.
634   process(clk)
635   begin
636     if rising_edge(clk) then
637       -- Latch the inputs
638       X_temp    <= X(X'left) & X(X'left) & X & "0000";
639       Y_temp    <= Y(Y'left) & Y(Y'left) & Y & "0000";
640       Func_temp <= Func;
641
642       -- Set the pipelined values to the previous values.
643
644       -- First quarter
645       FuncPipeline1       <= Func_temp;
646       CordSystemPipeline1 <= CordSystem;
647       ModePipeline1       <= Mode;
648
```

```
649       XPipeline1 <= XResultBus(3);
650       YPipeline1 <= YResultBus(3);
651       ZPipeline1 <= ZResultBus(3);
652
653
654       -- Second quarter
655       FuncPipeline2       <= FuncPipeline1;
656       CordSystemPipeline2 <= CordSystemPipeline1;
657       ModePipeline2       <= ModePipeline1;
658
659       XPipeline2 <= XResultBus(7);
660       YPipeline2 <= YResultBus(7);
661       ZPipeline2 <= ZResultBus(7);
662
663       -- Third quarter
664       FuncPipeline3       <= FuncPipeline2;
665       CordSystemPipeline3 <= CordSystemPipeline2;
666       ModePipeline3       <= ModePipeline2;
667
668       XPipeline3 <= XResultBus(11);
669       YPipeline3 <= YResultBus(11);
670       ZPipeline3 <= ZResultBus(11);
671
672       end if;
673   end process;
674
675   -- Log on the rising edge.
676   process(clk)
677   begin
678     if rising_edge(clk) then
679
680       Log (CordicLogID, YELLOW & "Rising Edge" & ANSI_RESET, DEBUG);
681       Log (CordicLogID, "Func: "    & to_string(Func), DEBUG);
682       Log (CordicLogID, "X: "      & to_string(Q1_14ToReal(X)), DEBUG);
683       Log (CordicLogID, "Y: "      & to_string(Q1_14ToReal(Y)), DEBUG);
684       Log (CordicLogID, "X_temp: " & to_string(Q1_14ToReal(X_temp)), DEBUG);
685       Log (CordicLogID, "Y_temp: " & to_string(Q1_14ToReal(Y_temp)), DEBUG);
686
687       if (Func /= "UUUUU") then
688         Log (CordicLogID, to_string(LF), DEBUG);
689         Log (CordicLogID, RepeatChar('-', 120), DEBUG);
690         PrintFunc(X, Y, Func);
691
692         if (Mode /= 'U') then
693           Log (CordicLogID, "Mode: "       & MODE_STRINGS(to_integer(Mode)), DEBUG);
694           Log (CordicLogID, "CordSystem: " & CORD_STRINGS(to_integer(unsigned(CordSystem))), DEBUG);
695         end if;
696
697       Log(CordicLogID,
698           HT & HT & HT & "X_A" & HT & HT & HT &  "X_B" & HT & HT & HT & "Y_A" &
699           HT & HT & HT & "Y_B" & HT & HT & HT & "Z_A"  & HT & HT & HT & "Z_B" &
700           HT & HT & "d_i" & HT
701           , DEBUG);
702
```

```
      for i in 0 to ITERATIONS - 1 loop
        Log(CordicLogID,
          "i=" & to_string(i) &
          HT & HT & to_string(Q3_18ToReal(XInputBus(i)), 9) &
          HT & HT & to_string(Q3_18ToReal(std_logic_vector(signed(YInputBus(i)) sra i)), 9) &
          HT & HT & to_string(Q3_18ToReal(YInputBus(i)), 9) &
          HT & HT & to_string(Q3_18ToReal(std_logic_vector(signed(XInputBus(i)) sra i)), 9) &
          HT & HT & to_string(Q3_18ToReal(ZInputBus(i)), 3) &
          HT & HT & to_string(Q3_18ToReal(ZConstantBus(i)), 3) &
          HT & HT & to_string(DBus(i)) & HT & HT ,
          DEBUG);
      end loop;

      Log(CordicLogID, "XResultBus(15): " & to_string(Q3_18ToReal(XResultBus(15))), DEBUG);
      Log(CordicLogID, "YResultBus(15): " & to_string(Q3_18ToReal(YResultBus(15))), DEBUG);
      Log(CordicLogID, "ZResultBus(15): " & to_string(Q3_18ToReal(ZResultBus(15))), DEBUG);
      Log(CordicLogID, "Returning: " & to_string(Q1_14ToReal(ResultComb)), DEBUG);
      Log(CordicLogID, "Returning: " & to_string((ResultComb)), DEBUG);
      Log(CordicLogID, RepeatChar('-', 120), DEBUG);
      Log(CordicLogID, to_string(LF), DEBUG);

    end if;
  end if;
end process;

-- Log on the falling_edge.
process(clk)
begin

  if falling_edge(clk) then
    Log(CordicLogID, YELLOW & "Falling Edge" & ANSI_RESET, DEBUG);
    PrintFunc(X, Y, Func);
    Log (CordicLogID, "X: "      & to_string(Q1_14ToReal(X)), DEBUG);
    Log (CordicLogID, "Y: "      & to_string(Q1_14ToReal(Y)), DEBUG);
    Log (CordicLogID, "X_temp: " & to_string(Q1_14ToReal(X_temp)), DEBUG);
    Log (CordicLogID, "Y_temp: " & to_string(Q1_14ToReal(Y_temp)), DEBUG);

    if (Mode /= 'U') then
      Log(CordicLogID, "Mode: "       & MODE_STRINGS(to_integer(Mode)), DEBUG);
      Log(CordicLogID, "CordSystem: " & CORD_STRINGS(to_integer(unsigned(CordSystem))), DEBUG);
    end if;

    Log(CordicLogID,
        HT & HT & HT & "X_A" & HT & HT & HT &  "X_B" & HT & HT & HT & "Y_A" &
        HT & HT & HT & "Y_B" & HT & HT & HT & "Z_A"  & HT & HT & HT & "Z_B" &
        HT & HT & "d_i" & HT , DEBUG);

    for i in 0 to ITERATIONS - 1 loop
      Log(CordicLogID,
        "i=" & to_string(i) &
        HT & HT &  to_string(Q3_18ToReal(XInputBus(i)), 9) &
        HT & HT & to_string(Q3_18ToReal(std_logic_vector(signed(YInputBus(i)) sra i)), 9) &
        HT & HT & to_string(Q3_18ToReal(YInputBus(i)), 9) &
        HT & HT & to_string(Q3_18ToReal(std_logic_vector(signed(XInputBus(i)) sra i)), 9) &
```

```
757           HT & HT & to_string(Q3_18ToReal(ZInputBus(i)), 9) &
758           HT & HT & to_string(Q3_18ToReal(ZConstantBus(i)), 9) &
759           HT & HT & to_string(DBus(i)) & HT & HT ,
760           DEBUG);
761       end loop;
762
763       Log(CordicLogID, "XResultBus(15): " & to_string(Q3_18ToReal(XResultBus(15))), DEBUG);
764       Log(CordicLogID, "YResultBus(15): " & to_string(Q3_18ToReal(YResultBus(15))), DEBUG);
765       Log(CordicLogID, "ZResultBus(15): " & to_string(Q3_18ToReal(ZResultBus(15))), DEBUG);
766       Log(CordicLogID, RepeatChar('-', 120), DEBUG);
767       Log(CordicLogID, to_string(LF), DEBUG);
768       Log(CordicLogID, "ZResultBus(15): " & to_string(ZResultBus(15)), DEBUG);
769       Log(CordicLogID, "ZResultBus(15) in Q1_14 format: " & to_string(Q3_18_To_Q1_14(ZResultBus(15))), DEBUG);
770       Log(CordicLogID, "ZResultBus(15) converted: " &
771         to_string(Q1_14ToReal(Q3_18_To_Q1_14(ZResultBus(15)))), DEBUG);
772       Log(CordicLogID, "Returning: " & to_string(Q1_14ToReal(ResultComb)), DEBUG);
773       Log(CordicLogID, "Returning: " & to_string((ResultComb)), DEBUG);
774       PrintFuncTemp(Func_temp);
775
776     end if;
777   end process;
778
779 end sim;
780
781
```